

Reasoning-aware Adaptive Routing for Hybrid GraphRAG

Nguyen Dinh Thanh

School of Information Technology and Communication,
Hanoi University of Science and Technology,

Hai Ba Trung, Hanoi, Vietnam

`thanh.nd225670@sis.hust.edu.vn`

June 8, 2026

Abstract Retrieval-Augmented Generation improves the factual reliability of large language models by grounding generated answers in external knowledge sources. In Vietnamese legal question answering, however, a fixed retrieval strategy is often insufficient. Some questions can be answered through direct textual lookup, while others require reasoning over legal relations such as amendment, repeal, replacement, cross-document reference, effective date, issuing authority, and article membership. Moreover, when a query is ambiguous or underspecified, immediate retrieval may ground the answer in the wrong legal document, article, entity, or time period.

This paper proposes a Vietnamese legal question answering system based on Hybrid GraphRAG with a two-stage Reasoning-aware Adaptive Router. Given a user query and an optional conversation history, the router selects one of four actions: dense retrieval, graph traversal, hybrid reasoning, or clarification. Stage 1 uses an XGBoost classifier over query-level reasoning and ambiguity features to make fast routing decisions. Stage 2 invokes an LLM-based verifier only for uncertain, ambiguous, or relation-heavy

cases. The retrieval backend combines a dense vector index over Vietnamese legal text chunks with a Neo4j legal knowledge graph containing legal documents, legal articles, vector chunks, agencies, concepts, and legal relations.

Experiments on a strict 600-query Vietnamese legal QA test set show that the Single-stage Router improves answer F1 to 0.4231, compared with 0.3626 for Pure Vector and 0.3556 for Pure Graph, while reaching 0.9350 routing accuracy. The Two-stage Hybrid system obtains the best answer F1, 0.4235, but slightly reduces strict routing accuracy to 0.9283 because the strict test set contains no intended clarification queries. On the original constructed ambiguity benchmark, the initial Stage 2 configuration increases clarification F1 to 0.684 with precision 1.000. After adding deterministic history-referent resolution and a severe-ambiguity override policy, the same ambiguity benchmark improves to clarification F1 = 0.840. On a separate 160-query conversation stress test, the router reaches clarification F1 = 0.862 and history-resolution accuracy = 0.850. These diagnostic results do not replace the strict benchmark; they show that ambiguity handling improves while multi-interpretation

cases and false clarification calibration remain open issues.

Keywords: Retrieval-Augmented Generation, GraphRAG, Hybrid RAG, Adaptive Routing, Vietnamese Legal QA, Knowledge Graphs, Multi-hop Reasoning, Ambiguity Detection.

1 Introduction

Large language models (LLMs) have demonstrated strong language understanding and generation ability, but their use in knowledge-intensive question answering remains limited when answers must be grounded in external, updateable, and verifiable evidence rather than relying only on model parameters [12, 7]. Retrieval-Augmented Generation (RAG) addresses this limitation by retrieving relevant passages from an external corpus and conditioning the generator on the retrieved evidence, which makes the system more suitable for domains where factual knowledge changes over time [11, 1]. However, conventional dense-retrieval RAG represents a corpus mainly as independent text chunks and is therefore more effective for single-hop or direct lookup questions than for questions requiring explicit reasoning over relations among documents, entities, or conditions [21, 17]. This limitation is especially important in legal question answering, where a user query may require identifying a specific article, following a cross-document reference, checking whether a regulation has been amended or repealed, or reasoning over issuing authorities and effective dates [2, 25, 19]. In Vietnamese legal QA, this problem is further complicated by Vietnamese-specific preprocessing issues such as word segmentation and multi-syllable named entities, which affect entity detection and legal reference matching [15, 22].

Recent work has explored several directions

to improve RAG. Self-RAG introduces reflection signals to decide when retrieval is needed and whether retrieved passages are useful, while FLARE performs active retrieval during generation when missing information is anticipated [1, 9]. RAPTOR organizes documents into hierarchical summaries for multi-level retrieval, improving retrieval over long or complex document collections [18]. Graph-based retrieval and GraphRAG further address relational retrieval by representing documents, entities, and relations as graph structures, allowing retrieval to follow explicit links rather than relying only on vector similarity [5, 24, 13]. Adaptive-RAG shows that different questions may require different retrieval and reasoning strategies depending on question complexity, while FrugalGPT and RouteLLM show that routing can reduce inference cost by assigning inputs to suitable models or inference paths [8, 3, 16]. Ambiguity-aware QA research also shows that underspecified or context-dependent questions may require rewriting or clarification before reliable answering [14, 6]. The difficulty is that these actions have different cost and reasoning profiles that cannot be reduced to a single routing dimension: dense retrieval is fast but relationally blind, graph traversal is more expressive but computationally heavier, hybrid reasoning combines both evidence sources at higher cost, and clarification introduces an additional conversational turn before retrieval. Nevertheless, existing directions do not directly address the four-way decision needed in Vietnamese Legal Hybrid GraphRAG: whether a query should use dense retrieval, graph traversal, hybrid reasoning, or clarification before retrieval.

Motivated by this gap, this paper proposes a Reasoning-aware Adaptive Router for Vietnamese Legal Hybrid GraphRAG. The central

challenge is that legal queries vary in both reasoning depth and ambiguity: a short query may still require multi-hop legal reasoning, while a long query may only need direct article lookup; similarly, an underspecified query can lead the system to retrieve the wrong document, article, or time period if clarification is not performed first. The motivation of this work is therefore to make retrieval strategy selection an explicit intermediate decision before answer generation. Given a user query and optional conversation history, the proposed router first extracts reasoning and ambiguity features, then uses a fast Stage 1 XGBoost classifier to predict an initial route, and finally invokes a Stage 2 LLM verifier only for uncertain, ambiguous, or relation-heavy cases [4, 8]. The main contribution is a practical, ambiguity-aware, and cost-aware routing mechanism that selects among `dense_retrieval`, `graph_traversal`, `hybrid_reasoning`, and `clarify` for Vietnamese legal question answering.

2 Related Work

The proposed system is related to six research lines: retrieval-augmented generation, graph retrieval and GraphRAG, adaptive routing, ambiguity handling, dataset and benchmark construction, and Vietnamese legal NLP.

2.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation combines parametric knowledge from neural language models with non-parametric knowledge retrieved from an external corpus [12, 7]. In a standard RAG pipeline, the input query is encoded into a dense representation, relevant passages are retrieved from a vector index, and the generator produces an answer conditioned on the retrieved context [11]. Dense passage retrieval is effective

for open-domain QA because it can retrieve semantically relevant passages even when lexical overlap is limited. However, it usually treats the corpus as a collection of independent chunks and is therefore less suitable for questions requiring explicit reasoning over document structure, entity relations, or multi-hop evidence chains [21, 17].

Recent RAG variants attempt to improve retrieval control and evidence usage. Self-RAG learns to retrieve, generate, and critique through self-reflection [1]; FLARE actively retrieves additional evidence during generation [9]; and RAPTOR constructs tree-organized summaries for retrieval at different abstraction levels [18]. These methods improve factual grounding, but they do not explicitly formulate the problem of choosing among dense retrieval, graph traversal, hybrid reasoning, and clarification in a legal Hybrid GraphRAG system.

2.2 Graph Retrieval and GraphRAG

Graph-based retrieval represents knowledge as nodes and edges, enabling retrieval to follow explicit relationships rather than relying only on vector similarity [20, 24]. Microsoft GraphRAG builds an entity graph and community summaries for query-focused summarization over private corpora [5]. Knowledge Graph Prompting constructs a graph over multiple documents and uses graph traversal to gather supporting passages for multi-document QA [24]. GraphReader structures long documents as a graph and allows an agent to explore relevant nodes and neighbors [13]. HippoRAG proposes a neurobiologically inspired graph memory for RAG, using a hippocampus-like indexing scheme to support multi-hop retrieval over long-term knowledge [10]. While this approach improves associative retrieval, it does not address the rout-

ing decision of when graph traversal is necessary versus direct dense lookup.

For legal QA, graph-based retrieval is particularly relevant because legal documents naturally contain structured relations such as amendment, repeal, replacement, reference, jurisdiction, issuing authority, article membership, and effective date [19]. However, graph construction and traversal introduce additional computational cost because the system must perform entity linking, relation matching, graph querying, and evidence serialization. Therefore, graph retrieval should be used selectively rather than applied to every query.

2.3 Adaptive Routing and Cost-aware Inference

Adaptive routing aims to choose the most appropriate model, retriever, or reasoning strategy for each input. Adaptive-RAG proposes selecting QA strategies based on question complexity, showing that not all questions require the same retrieval or reasoning procedure [8]. FrugalGPT and RouteLLM demonstrate that routing can reduce inference cost while maintaining quality when different models or inference paths have different capabilities and costs [3, 16]. These works motivate the cost-aware aspect of our router. However, they mainly focus on routing among model sizes, retrieval depths, or QA strategies, rather than explicitly routing among dense retrieval, graph traversal, hybrid reasoning, and clarification for legal QA.

2.4 Ambiguity Handling in QA

Real user questions are often ambiguous, incomplete, or dependent on previous conversational turns. CANARD studies question rewriting for context-dependent questions [6], while AmbigQA shows that open-domain questions may have multiple valid interpretations that

should be handled explicitly [14]. In high-stakes domains such as law, ambiguity is especially problematic because answering an underspecified question may retrieve evidence for the wrong document, article, agency, or time period. Therefore, our system treats `clarify` as a valid routing action rather than an exceptional failure case.

2.5 Dataset and Benchmark Construction

Recent benchmark papers emphasize that a dataset contribution should report not only final model scores, but also source provenance, generation pipeline, label definition, quality analysis, evaluation protocol, and scope limitations. MIST, although developed for a different modality and task, is an example of this reporting style because it describes the data generation pipeline, statistics, quality analysis, and task-specific metrics [23]. We follow this style lightly for Vietnamese legal QA: the present dataset is organized around data sources, construction rules, routing labels, split protocol, and limitations, but it is not compared as a method against MIST.

2.6 Vietnamese NLP and Legal QA

Vietnamese NLP introduces additional challenges because word boundaries are not always marked by whitespace and many entity names consist of multi-syllable expressions. VnCoreNLP provides Vietnamese word segmentation, part-of-speech tagging, named entity recognition, and dependency parsing [22], while PhoBERT provides pretrained Vietnamese representations for downstream NLP tasks [15]. Legal NLP further requires domain-specific reasoning over formal documents, citations, and regulatory relations [2, 25]. The proposed work differs from general-purpose RAG systems by placing

reasoning-aware routing at the center of Vietnamese legal question answering.

2.7 Comparative Discussion

Table 1 summarizes the relationship between prior research lines and the proposed system. Existing RAG methods provide a strong foundation for grounding LLM outputs in retrieved passages, but they mainly operate over unstructured text chunks and do not explicitly encode legal relations. Graph-based retrieval methods improve relational evidence access, but they often assume that graph retrieval is the main retrieval mechanism instead of deciding when graph traversal is necessary. Adaptive routing methods demonstrate that inference paths can be selected dynamically, but most of them focus on routing among model sizes, retrieval depths, or QA strategies rather than routing among dense retrieval, graph traversal, hybrid reasoning, and clarification. Ambiguity-aware QA methods show the importance of rewriting or clarifying underspecified questions, but clarification is rarely integrated as a retrieval route in Hybrid GraphRAG.

The proposed system differs from these lines by treating route selection as a reasoning-aware decision before retrieval. In Vietnamese legal QA, this design is necessary because the same surface form may correspond to different legal reasoning requirements. A query asking about a named article may be answered by dense retrieval, whereas a query asking whether a decree amends another document requires graph traversal. A query that combines article content with amendment or effective-date relations may require hybrid reasoning. If the legal target is missing, the safest action is clarification rather than retrieval.

Table 1: Summary of related research lines.

Line	Limitation for this work	Role
Dense RAG [12, 11, 7]	Retrieves relevant text but does not explicitly model legal relations such as amendment, repeal, reference, or effective date.	Dense route
Adv. RAG [1, 9, 18]	Improves retrieval control or hierarchy, but does not route among dense, graph, hybrid, and clarify actions.	Control
Graph RAG [5, 24, 13, 10]	Supports relational retrieval, but graph traversal can be unnecessary for simple legal lookup questions.	Graph route
Routing [8, 3, 16]	Routes models or retrieval depths, but not legal evidence mechanisms and clarification decisions.	Router
Ambiguity QA [14, 6]	Handles ambiguous questions, but clarification is rarely integrated as a Hybrid GraphRAG route.	Clarify
VN Legal NLP [15, 22, 2, 25]	Provides language or legal NLP foundations, but not reasoning-aware retrieval routing.	Features
Ours	Selects the evidence mechanism according to reasoning, ambiguity, and cost constraints.	4-way route

3 Methodology

3.1 Problem Definition

Let $\mathcal{C} = \{D_1, D_2, \dots, D_N\}$ denote the Vietnamese legal corpus, where D_i is the i -th legal document and N is the number of documents. Each document contains legal text and metadata such as title, legal type, issuing authority, source identifier, article structure, and available temporal information. From $\mathcal{C}$, the system builds two complementary knowledge sources. The first source is a dense vector index $\mathcal{I}_v = \{(c_j, \mathbf{e}_j)\}_{j=1}^M$, where c_j is the j -th legal text chunk, $\mathbf{e}_j \in \mathbb{R}^d$ is its d -dimensional embedding, and M is the number of indexed chunks. The second source is a legal knowledge graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the set of graph nodes and $\mathcal{E}$ is the set of directed typed edges. A node $v \in \mathcal{V}$ may represent a legal document, legal article, vec-

tor chunk, agency, or legal concept. An edge $(u, r, v) \in \mathcal{E}$ represents a legal relation r from source node u to target node v , for example HAS_ARTICLE, BELONGS_TO, REFERENCES, AMENDS, REPEALS, or REGULATES_CONCEPT.

At inference time, the input is the current user query q_t and an optional conversation history $H = \{(q_1, a_1), \dots, (q_{t-1}, a_{t-1})\}$. Here, t denotes the current dialogue turn, q_i denotes the user query at turn i , and a_i denotes the corresponding system answer. If the current query is the first turn, then $H = \emptyset$. The router must choose one action from the route set

$$\mathcal{D} = \{d_v, d_g, d_h, d_c\}, \quad (1)$$

where d_v denotes `dense_retrieval`, d_g denotes `graph_traversal`, d_h denotes `hybrid_reasoning`, and d_c denotes `clarify`. The routing function is defined as

$$\rho : (q_t, H) \rightarrow d^*, \quad d^* \in \mathcal{D}, \quad (2)$$

where d^* is the final selected route.

If $d^* \neq d_c$, the system retrieves an evidence set E_{d^*} and generates a grounded answer. For the three retrieval routes, the evidence set is

$$E_{d^*} = \begin{cases} E_v = \text{TopK}(q_t, \mathcal{I}_v, k), & d^* = d_v, \\ E_g = \text{Traverse}(q_t, \mathcal{G}, h), & d^* = d_g, \\ E_v \cup E_g, & d^* = d_h, \end{cases} \quad (3)$$

where k is the dense retrieval depth and h is the maximum graph traversal depth. The generator then produces

$$a_t = \text{Generate}(q_t, H, E_{d^*}). \quad (4)$$

If $d^* = d_c$, the system does not retrieve evidence and instead returns

$$c_t = \text{Clarify}(q_t, H), \quad (5)$$

where c_t asks the user to specify the missing legal document, article, entity, time period, or conversational referent. This distinction is important in legal QA because an answer grounded in the wrong legal target can be more harmful than asking a follow-up question.

3.2 Dataset Construction

Following the benchmark-reporting style discussed in Section 2, we describe the dataset by its provenance, construction pipeline, label rules, split protocol, and limitations rather than only by final counts. The routing dataset is self-constructed from processed Vietnamese legal sources. The first source contains legal documents stored as JSONL records with document identifiers, titles, document types, issuing authorities, and normalized legal text. The second source is the Vietnamese electronic codification system, originally organized by theme, topic, and article. Because retrieval and graph construction require stable evidence units, codification records are flattened into article-level objects with fields such as `doc_id`, `title`, `theme`, `topic`, `content_markdown`, and source meta-data.

The construction pipeline has four stages. First, legal documents are normalized and segmented into article-level records. Second, QA samples are linked to one or more legal articles through `article_key`. Third, these article keys are parsed into a structured `relevant_articles` list, joined with the article corpus to create `gold_context`, and converted into `supporting_facts`. Finally, route labels are derived from the supporting evidence structure and a strict train/dev/test split is produced.

Each QA sample contains a Vietnamese legal question, a reference answer, an evidence text, a legal source identifier, and an `article_key`

connecting the sample to one or more supporting legal articles. The field `article_key` follows the format `DOC_NUMBER::ARTICLE_ID`; for example, `49/VBHN-VPQH::Điều 75`. Low-quality samples are removed with deterministic filters: unsupported route labels are excluded, questions shorter than 12 characters or answers shorter than two characters are discarded, duplicate questions are removed, and placeholder answers such as *không có trong ngữ cảnh*, *không có thông tin*, or *đang cập nhật nội dung* are filtered out.

Routing labels are derived from evidence structure. A sample is labeled `dense_retrieval` when it can usually be answered from a single article or direct textual evidence. It is labeled `graph_traversal` when it requires multiple articles from the same legal document, indicating intra-document structural or relational reasoning. It is labeled `hybrid_reasoning` when the supporting evidence spans multiple documents, because these cases often require both semantic retrieval and relation-aware graph reasoning. These labels are useful for system development, but they are metadata-derived rather than independently human-annotated; this limitation is discussed in Section 6.

The strict routing dataset contains 1,126 samples with three observed retrieval labels. It is split into train, development, and test sets using stratified sampling with a fixed random seed. To reduce leakage, the test set is sampled first and is never used for router training or threshold tuning; the development set is then sampled from the remaining pool, and the remaining samples form the training set. Table 2 reports the split statistics.

Because the strict split does not contain standalone `clarify` training labels, clarification is evaluated on a separate constructed ambiguity benchmark. This synthetic stress-test set con-

Table 2: Vietnamese legal QA routing dataset split by route label.

Split	Dense	Graph	Hybrid	Total
Train	356	54	36	446
Dev	50	20	10	80
Test	300	150	150	600
Total	706	224	196	1,126

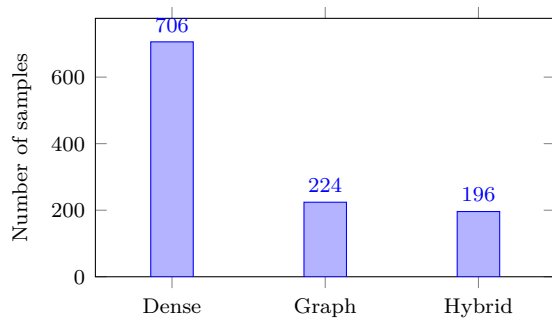


Figure 1: Route-label distribution of the strict Vietnamese legal QA routing dataset. The distribution is skewed toward dense retrieval, so Macro-F1 and per-class analysis are needed in addition to accuracy.

tains 234 queries: 156 ambiguous queries labeled **clarify** and 78 answerable queries labeled as one of the three retrieval routes. The ambiguous cases are organized into four types: incomplete context (39), missing entity (39), unresolved pronoun reference (42), and multiple interpretation (36). This separation is deliberate: the strict split evaluates retrieval-mechanism selection, whereas the ambiguity benchmark evaluates whether the router can avoid unsafe answering when the legal target is underspecified. Since the ambiguity set is template-generated rather than independently human-annotated, it should be interpreted as a controlled stress test, not as an external public benchmark.

After Phase 3, we also construct a 160-query conversation-aware ambiguity stress test. This diagnostic set contains eight equally sized categories: **answerable_with_history**, **clarify_without_history**, **irrelevant_history**, **conflicting_history**, **missing_entity**, **multi_interpretation**, **clear_dense_control**, and **clear_graph_or_hybrid_control**. Its purpose is not to replace the strict 600-query end-to-end benchmark, but to test whether the router uses conversation history correctly: a resolved referent should allow retrieval, while absent, irrelevant, or conflicting history should preserve or force clarification. The benchmark is template-controlled and should therefore be interpreted as a stress test for router behavior rather than as a production distribution.

3.3 System Pipeline

Figure 2 summarizes the proposed pipeline. The input (q_t, H) is first converted into query-level reasoning and ambiguity features. In the conversation-aware implementation, a deterministic **HistoryResolver** first produces r_H so

Table 3: Dataset construction protocol for the primary evaluation sets used in this study.

Item	Strict legal QA set	Ambiguity stress test
Purpose	Evaluate retrieval-route selection and end-to-end QA quality.	Evaluate whether the router asks for clarification before unsafe retrieval.
Source	Processed Vietnamese legal QA samples linked to legal articles by article_key .	Deterministic templates for underspecified legal questions plus clear answerable controls.
Labels	Dense, graph, and hybrid labels derived from evidence structure.	clarify positives and dense/graph/hybrid negative controls.
Size	1,126 total samples; 600 held out for strict testing.	234 total queries; 156 clarify and 78 answerable controls.
Quality control	Deduplication, short-answer filtering, placeholder-answer filtering, and held-out test-first split.	Fixed generation rules, balanced control routes, and explicit ambiguity-type labels.
Scope	Tests retrieval strategy selection, not standalone clarification.	Controlled stress test; requires human review before being treated as a production benchmark.

that the feature extractor can distinguish resolved, missing, irrelevant, and conflicting history. Stage 1 predicts an initial route using a fast feature-based model. Stage 2 is invoked only when the trigger function detects low confidence, ambiguity, or high reasoning demand. The final route then controls whether the system executes dense retrieval, graph traversal, hybrid retrieval, or clarification.

3.4 Running Example

This subsection illustrates how the formal routing definition and Figure 2 are used for different Vietnamese legal questions. Consider the query $q_1 = \text{“Điều kiện để được cấp giấy phép xây dựng là gì?”}$. This query expresses a direct legal information need. It asks for conditions that are usually stated in one or a small number of legal provisions, and it does not explicitly mention amendment, repeal, replacement, cross-document reference, or effective-date reasoning. The feature extractor therefore assigns low ambiguity and low multi-hop signals. In this case, Stage 1 is expected to predict the dense route d_v with high confidence, so Stage 2 is bypassed and the system retrieves relevant chunks from the dense index $\mathcal{I}_v$. The evidence set is $E_{d^*} = E_v = \text{TopK}(q_1, \mathcal{I}_v, k)$, and the answer generator produces a grounded answer from the retrieved legal text.

In contrast, consider $q_2 = \text{“Nghị định nào sửa đổi quy định về xử phạt vi phạm hành chính trong lĩnh vực đất đai, và điều khoản hiệu lực của nghị định đó được quy định ở đâu?”}$. This query contains explicit relation signals such as *sửa đổi* and *hiệu lực*. It also requires connecting at least two legal objects: the document being amended and the decree that introduces the amendment. Moreover, the phrase asking where the validity clause is specified requires locating

an article or provision inside the modifying decree. The feature extractor therefore assigns stronger legal-relation and multi-hop signals. If the answer can be obtained mainly from legal relations, the router selects d_g and retrieves graph paths from $\mathcal{G}$. If both relation paths and article text are needed, the router selects d_h and constructs $E_{d^*} = E_v \cup E_g$. This example shows why graph traversal should be available but should not be applied to every legal query.

Finally, consider $q_3 = \text{“Văn bản đó còn hiệu lực không?”}$. The phrase *văn bản đó* is an unresolved referent when the conversation history H does not identify a unique legal document. If the system retrieves evidence immediately, it may answer about the wrong document or time period. Therefore, the ambiguity score $\phi_{\text{amb}}(q_3, H)$ should be high, the Stage 2 verifier may be triggered, and the final route should be d_c . Instead of producing an answer, the system returns a clarification question such as *“Bạn muốn hỏi hiệu lực của văn bản nào?”*. This behavior is important in legal QA because asking for clarification is safer than generating an answer from an incorrectly inferred legal target.

3.5 Legal Knowledge Base Construction

The retrieval backend uses `microsoft/Harrier-OSS-v1-0.6B` embeddings and represents 199,530 graph-linked vector chunks as Neo4j VECTORCHUNK nodes. Dense retrieval is appropriate for direct questions whose answers are stated in legal text, such as definitions, conditions, penalties, or procedures. The graph backend stores explicit relations among legal documents, articles, chunks, and legal concepts. This graph is necessary because many legal questions require following explicit legal links, such as article membership, references,

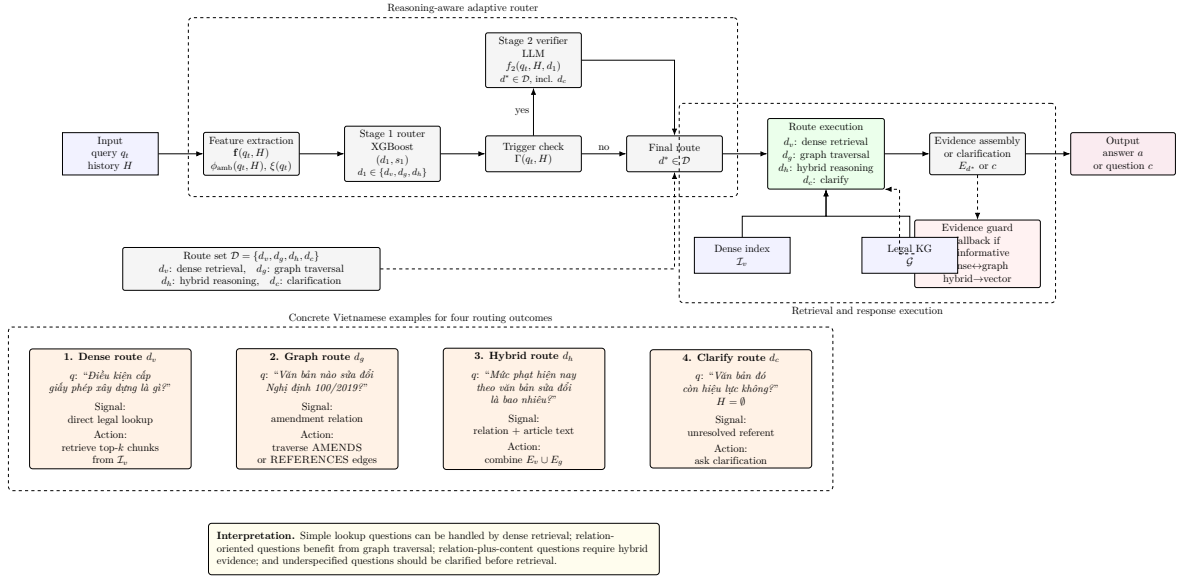


Figure 2: Overview of the proposed Reasoning-aware Adaptive Router for Vietnamese Legal Hybrid GraphRAG. The figure also explains the notation used in the paper: q_t is the current query, H is the conversation history, f is the extracted feature vector, ϕ_{amb} is the ambiguity score, ξ is the multi-hop score, (d_1, s_1) is the Stage 1 route and confidence, and d^* is the final route. The dashed fallback guard summarizes the implementation policy that retries another retrieval backend when the first answer is uninformative.

amendments, repeal, and concept regulation.

Table 4 reports the current graph after the Pháp điển migration and vector-chunk linking. Compared with the earlier graph snapshot, the most important schema fix is that Pháp điển article nodes are now exposed as LEGALARTICLE nodes with content and connected to parent legal documents through HAS_ARTICLE. This change is important because article-level retrieval is the natural evidence unit for legal QA.

Table 4: Statistics of the enriched Vietnamese legal graph $\mathcal{G}$ after graph migration. Vector chunks are reported using the post-migration graph-linked VECTORCHUNK count.

Graph item	Count
Nodes $ \mathcal{V} $	419,251
Edges $ \mathcal{E} $	1,239,542
Legal documents	219,439
Legal articles	70,347
Neo4j VECTORCHUNK nodes	199,530
Legal concepts	10
Average degree	5.91
Isolated nodes	3,102

Table 4 and Table 5 report the current graph statistics. The HAS_ARTICLE count of 70,347 reflects the graph migration step that promoted Pháp điển article nodes from LEGALDOC to LEGALARTICLE; the BELONGS_TO relation connects vector chunks back to their source legal documents, which is important for hybrid retrieval and evidence serialization.

3.6 Reasoning-aware Feature Extraction

For each input (q, H) , the router extracts a feature vector $\mathbf{f}(q, H)$ and two auxiliary scores: the ambiguity score $\phi_{\text{amb}}(q, H)$ and the multi-hop score $\xi(q)$. The feature vector combines

Table 5: Main relation types in the enriched legal graph.

Relation type	Count
REFERENCES	769,000
HAS_VECTOR_CHUNK	199,802
BELONGS_TO	199,530
HAS_ARTICLE	70,347
REGULATES_CONCEPT	249
NEXT_ARTICLE	239
CROSS_REFERENCES	133
MENTIONS_DOC	84
INTRA_DOC_REFERENCE	83
AMENDS	27
GUIDES	25
REPEALS	19
IMPLEMENTS	3

statistical features, Vietnamese legal-reference patterns, entity features, question-type cues, relation cues, and context-dependence signals. Statistical features include query length, token count, named-entity count, entity density, and the number of legal identifiers. Question-type features encode Vietnamese question forms such as *gì, khi nào, ở đâu, bao nhiêu*, and *như thế nào*. Legal-relation features detect phrases such as *sửa đổi, bãi bỏ, thay thế, có hiệu lực*, and *được quy định tại*. Multi-hop features capture comparison markers, conditional clauses, and signals that more than one article or document may be required. Ambiguity features detect unresolved pronouns, vague legal targets, missing entities, missing temporal scopes, and queries that depend on absent conversation history.

Phase 3 adds a deterministic **HistoryResolver** before ambiguity scoring. It extracts legal referent candidates from the conversation history and returns

$$r_H = \text{Resolve}(q_t, H), \quad (6)$$

where r_H contains `candidate_referents`, an optional `resolved_referent`, a `history_resolution_status`, and a `history_resolution_confidence`. The possible status values are `resolved`, `no_history`, `irrelevant_history`, `conflicting_history`, and `not_needed`. This signal prevents the router from treating every non-empty history as useful context. If a query such as “*Văn bản đó còn hiệu lực không?*” has a unique document referent in history, the resolved referent can unblock graph traversal. If history is absent, irrelevant, or contains multiple competing referents, the same surface form remains ambiguous and can be routed to clarification.

3.7 Two-stage Adaptive Router

Stage 1 is an XGBoost classifier f_1 trained on the strict three-class routing data. It maps the extracted feature vector to an initial route and confidence score:

$$(d_1, s_1) = f_1(\mathbf{f}(q, H)), \quad (7)$$

where d_1 is the initial route and $s_1 \in [0, 1]$ is the confidence score. Because the Stage 1 training split contains only `dense_retrieval`, `graph_traversal`, and `hybrid_reasoning`, Stage 1 is not expected to learn the `clarify` class. This is an architectural property of the current training setup rather than an implementation bug.

Stage 2 is an LLM-based verifier f_2 invoked only for selected cases. Let $b = \phi_{\text{amb}}(q, H)$ be the ambiguity score. The trigger indicators are

$$\gamma_{\text{conf}} = \mathbb{I}[s_1 < \tau_{\text{conf}}], \quad (8)$$

$$\gamma_{\text{amb}} = \mathbb{I}[b > \tau_{\text{force}}], \quad (9)$$

$$\gamma_{\text{hop}} = \mathbb{I}[d_1 \neq d_g \wedge \xi(q) = 2]. \quad (10)$$

The trigger function is

$$\Gamma(q, H, r_H) = \mathbb{I}[\gamma_{\text{conf}} \vee \gamma_{\text{amb}} \vee \gamma_{\text{hop}}]. \quad (11)$$

The final route is then

$$d^* = \begin{cases} f_2(q, H, d_1), & \Gamma(q, H) = 1, \\ d_1, & \Gamma(q, H) = 0. \end{cases} \quad (12)$$

The current default implementation uses $\tau_{\text{conf}} = 0.50$ for low-confidence triggering, $\tau_{\text{force}} = 0.60$ for ambiguity-based Stage 2 forcing, and a separate clarification override threshold $\tau_{\text{clar}} = 0.80$. The ablation in Section 5.5 shows that τ_{conf} mainly changes how often Stage 2 is considered, while τ_{clar} has a stronger effect on false clarification in non-ambiguous routing data. The Phase 3 implementation additionally uses r_H : resolved history can block unnecessary clarification and restore a retrieval route, while `no_history`, `irrelevant_history`, or `conflicting_history` can force Stage 2 or clarification when the query contains a contextual reference.

Algorithm 1 Two-stage Adaptive Router

Require: Query q , history H , thresholds

$\tau_{\text{conf}}, \tau_{\text{force}}, \tau_{\text{clar}}$

Ensure: Final route $d^* \in \mathcal{D}$

- 1: $r_H \leftarrow \text{Resolve}(q, H)$
 - 2: Compute $\mathbf{f}(q, H, r_H)$, $\phi_{\text{amb}}(q, H, r_H)$, and $\xi(q)$
 - 3: $(d_1, s_1) \leftarrow f_1(\mathbf{f}(q, H, r_H))$
 - 4: Compute $\Gamma(q, H, r_H)$ using Eq. (11)
 - 5: **if** $\Gamma(q, H) = 1$ **then**
 - 6: $d^* \leftarrow f_2(q, H, d_1)$
 - 7: **else**
 - 8: $d^* \leftarrow d_1$
 - 9: **end if**
 - 10: **return** d^*
-

Algorithm 1 presents the routing logic in simplified form. The deployed implementation adds several practical heuristics beyond the three-indicator trigger: a high-confidence dense fast

Table 6: Stage 2 verification policy used by the implemented two-stage router.

Signal	Condition	Purpose
Low confidence	$s_1 < \tau_{\text{conf}}$	Check uncertain Stage 1 decisions.
Ambiguity risk	$b \geq \tau_{\text{force}}$	Avoid retrieval when the legal target is under-specified.
Resolved history	$r_H.\text{status} = \text{resolved}$	Use the referent and avoid unnecessary clarification.
Unresolved history	$r_H.\text{status}$ is no, irrelevant, or conflicting	Force verification or clarification.
Dense fast path	High-confidence dense, low ambiguity, no strong legal signal	Skip Stage 2 for clear lookup queries.
Dense legal signal	Dense route with legal-reference, graph, authority, or effect signals	Catch cases needing graph or hybrid retrieval.
Reasoning signal	Multi-hop, cross-document, complex, or relation-heavy features	Verify relation-heavy legal questions.
Clarify sanity check	Predicted <code>clarify</code> with low ambiguity score	Reduce false clarification on answerable questions.

path that skips Stage 2 when the query has no legal-reference or relational signal, a reasoning-signal check that can escalate queries with strong legal features even at moderate Stage 1 confidence, and a rescue-only override policy for Stage 2 that permits route changes only under narrowly defined conditions. These heuristics are configurable and are designed to reduce unnecessary Stage 2 invocations on clear queries while preserving verification for uncertain cases.

3.8 Route Execution

For d_v , the system retrieves the top- k chunks from the dense index $\mathcal{I}_v$ and sends the selected chunks to the answer generator. For d_g , the system links query entities to graph nodes and traverses relevant relations in $\mathcal{G}$, returning serialized graph paths and linked legal evidence. For d_h , the system combines semantic text evidence and graph evidence as $E_v \cup E_g$. For d_c , the system returns a clarification question and does not generate a final legal answer. This route execution design matches the main motivation of the project: not every query needs graph traversal, and not every query should be answered immediately. Table 7 summarizes the fallback behavior used by the implemented pipeline.

3.9 Implementation Details

The implementation uses a Chroma vector store for dense retrieval and Neo4j for graph retrieval. The main Vietnamese legal vector store is persisted at `data/vector_store/chroma_harrier_oss_0_6b`. Its chunks are embedded with `microsoft/Harrier-OSS-v1-0.6B`. In the graph, the corresponding chunk layer is represented by `VECTORCHUNK` nodes and connected back to source legal documents through `BELONGS_TO` edges. This connection is useful when hybrid retrieval must serialize both graph paths and source text evidence.

Table 7: Route execution and fallback policy in the implemented Hybrid GraphRAG pipeline.

Route	Primary execution	Fallback / guard
dense retrieval	Retrieve from Chroma and generate from dense text chunks.	If the answer is uninformative, optionally retry graph retrieval.
graph traversal	Retrieve graph context through the GraphRAG adapter and serialize legal paths/evidence.	If graph retrieval or generated evidence is uninformative, fallback to dense retrieval.
hybrid reasoning	Merge vector context and graph context, then generate from the combined evidence.	If hybrid synthesis is uninformative and vector evidence exists, fallback to dense retrieval.
clarify	Generate a follow-up clarification question.	No retrieval is executed; if LLM clarification fails, use the ambiguity detector’s fallback question.

Stage 1 uses an XGBoost classifier over the project’s hand-crafted reasoning and ambiguity features. The model is stored as a local router checkpoint and is used for fast feature-based routing at inference time. Stage 2 is served through an OpenAI-compatible LLM endpoint; in the reported experiments, the local configuration uses a Qwen3-compatible model for structured route verification. The verifier is prompted to return a route decision, confidence, optional clarification question, and reasoning trace, which are then parsed by the two-stage router.

Vietnamese named entity recognition uses NlpHUST/ner-vietnamese-electra-base for extracting candidate legal entities and agencies. Graph retrieval uses Neo4j full-text indexes over legal documents, legal articles, and vector chunks. These implementation choices are designed to keep Stage 1 lightweight while allowing Stage 2 and graph retrieval to be invoked only when the query requires additional reasoning or clarification.

4 Experimental Evaluation

4.1 Research Questions

The evaluation addresses four research questions. RQ1 asks whether adaptive routing improves answer quality compared with always using dense retrieval or always using graph retrieval. RQ2 asks whether the router selects retrieval routes accurately on the strict Vietnamese legal QA test set. RQ3 asks whether Stage 2 improves clarification behavior on ambiguous queries and what types of ambiguity it can detect. RQ4 asks whether the latency overhead of Stage 2 justifies selective verification rather than invoking the LLM verifier for every query.

4.2 Experimental Setup

The strict benchmark contains 600 held-out legal QA queries with route labels distributed as 300 dense, 150 graph, and 150 hybrid. It is used for end-to-end answer-quality and routing-accuracy evaluation. The ambiguity benchmark contains 234 queries and is used only to evaluate clarification behavior. The two benchmarks are kept separate because the strict test set is designed to evaluate retrieval strategy selection, while the ambiguity benchmark is designed to test whether the system can ask follow-up questions before unsafe retrieval.

The conversation-aware ambiguity stress test contains 160 queries and is reported as an additional diagnostic after Phase 3. It is separated from the strict benchmark because it evaluates a different behavior: whether conversation history resolves or fails to resolve contextual references. Therefore, its metrics are not used to replace the strict end-to-end QA results in Table 9.

Table 8: End-to-end systems evaluated on the strict Vietnamese legal QA benchmark.

System	Description
Pure Vector	Always uses dense retrieval from the Chroma index. This baseline measures the quality of text-only semantic retrieval.
Pure Graph	Always uses graph traversal in Neo4j. This baseline measures whether relation-only retrieval is sufficient for legal QA.
Single-stage Router	Uses only the Stage 1 XGBoost route prediction and then executes the selected dense, graph, hybrid, or clarify action.
Two-stage Hybrid	Uses Stage 1 plus the selective Stage 2 LLM verifier for uncertain, ambiguous, or reasoning-heavy cases. This is the proposed full system.

Answer quality is reported with Exact Match

(EM) and token-level F1. Because generated Vietnamese legal answers are often paraphrastic and longer than the reference answer, F1 is more informative than EM. Routing quality is reported with routing accuracy, Stage 2 trigger rate, Stage 2 override rate, and latency.

4.3 Metrics

For route classification, accuracy is the proportion of correct route decisions. For a route label ℓ , precision, recall, and F1 are defined as

$$P_\ell = \frac{TP_\ell}{TP_\ell + FP_\ell}, \quad (13)$$

$$R_\ell = \frac{TP_\ell}{TP_\ell + FN_\ell}, \quad (14)$$

$$F1_\ell = \frac{2P_\ell R_\ell}{P_\ell + R_\ell}. \quad (15)$$

Macro-F1 averages $F1_\ell$ equally over labels, while Weighted-F1 weights each label by support. Macro-F1 remains important because the dataset is imbalanced toward dense retrieval. For clarification, precision measures whether queries predicted as `clarify` are truly ambiguous, recall measures how many ambiguous queries are detected, and clarify F1 summarizes the trade-off.

5 Results and Discussion

5.1 End-to-end Answer Quality on the Strict Test Set

Table 9 reports the strict 600-query benchmark. Pure Vector obtains $F1 = 0.3626$, while Pure Graph obtains $F1 = 0.3556$. The graph-only system is slightly lower than the vector-only system, which suggests that graph traversal is not automatically better for every legal question. A graph route can provide broader relational context, but this context may also be diffuse when the query only requires direct textual lookup.

The Single-stage Router improves F1 to 0.4231, a relative gain of approximately 16.7% over Pure Vector. The Two-stage Hybrid system obtains $F1 = 0.4235$, which is the best score but only slightly above Single-stage Router on this strict non-ambiguous test set. Exact Match (EM) is near zero (0.0017) for all systems because Vietnamese legal answers are typically paraphrastic, longer than the reference text, and involve normalization variations. Since EM does not discriminate among systems in this setting, token-level F1 is used as the primary answer-quality metric.

Table 9: End-to-end QA results on the strict Vietnamese legal QA test set. EM is near zero for all systems because legal answers are paraphrastic; F1 is the primary metric.

System	F1	Routing Acc.	Latency
Pure Vector	0.3626	0.5000	1,270.7 ms
Pure Graph	0.3556	0.2500	2,283.4 ms
Single-stage Router	0.4231	0.9350	2,209.2 ms
Two-stage Hybrid	0.4235	0.9283	3,913.4 ms

The routing results support the main claim of adaptive retrieval. Pure Vector has routing accuracy 0.5000 because half of the strict test set is dense. Pure Graph has routing accuracy 0.2500 because only one quarter of the strict test set is graph. In contrast, Single-stage Router reaches 0.9350 routing accuracy, and Two-stage Hybrid reaches 0.9283. The small decrease from Single-stage Router to Two-stage Hybrid is not a bug. The strict benchmark contains no intended clarify queries, so Stage 2 occasionally changes a correct Stage 1 route into an unnecessary clarification or alternative retrieval route. This is why Stage 2 should be interpreted as an ambiguity verifier, not as a component that must improve strict routing accuracy on every benchmark.

Table 10 reports the offline Stage 1 classifier behavior on the strict test split. This diagnostic is useful because the strict routing data are imbalanced. The classifier is strongest on `dense_retrieval` and `hybrid_reasoning`, while `graph_traversal` is the hardest class. The offline classifier reaches accuracy 0.805 and Macro-F1 0.780; this diagnostic is separate from the deployed end-to-end routing metric in Table 9.

Table 10: Offline Stage 1 router per-class performance on the strict test split.

Class	Precision	Recall	F1	Support
Dense retrieval	0.825	0.897	0.859	300
Graph traversal	0.646	0.547	0.592	150
Hybrid reasoning	0.898	0.880	0.889	150
Macro average	0.790	0.774	0.780	600

To contextualize the Stage 1 XGBoost classifier, we compare it against several routing baselines trained and evaluated on the same strict test split. Table 11 reports the results. The MajorityRoute baseline always predicts the most frequent training class and achieves Macro-F1 = 0.222, confirming that simple frequency cannot solve routing. The KeywordRuleRouter uses hand-crafted regex features and reaches Macro-F1 = 0.497. Logistic Regression and Random Forest, trained on the same 28-dimensional feature vector as XGBoost, reach Macro-F1 0.781 and 0.790 respectively. The PhoBERT sequence classifier achieves the highest offline routing Macro-F1 = 0.901, outperforming XGBoost (0.739). However, XGBoost is chosen as the Stage 1 classifier for two reasons: (1) routing latency is approximately 8.9 ms for XGBoost versus 53.3 ms for PhoBERT per query, which matters when Stage 1 is a gateway for every incoming query; and (2) XGBoost runs on CPU

without a GPU dependency, simplifying deployment. PhoBERT as a Stage 1 replacement is a promising direction for future work where higher routing accuracy can justify the additional inference cost.

Table 11: Routing classifier comparison on the strict test split (600 queries). All classifiers use question text only; MetadataRuleRouter is oracle-style and excluded from comparison. PhoBERT achieves the best Macro-F1 but XGBoost is chosen for latency.

Classifier	Acc.	Macro-F1	W-F1
MajorityRoute	0.500	0.222	0.333
KeywordRuleRouter	0.612	0.497	0.568
Logistic Regression	0.825	0.781	0.806
Random Forest	0.833	0.790	0.815
XGBoost (Stage 1)	0.807	0.739	0.772
PhoBERT-base-v2	0.913	0.901	0.910

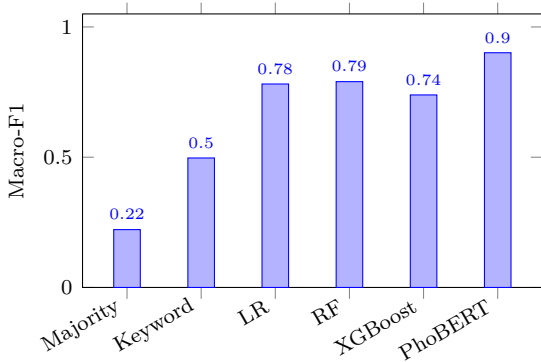


Figure 3: Routing classifier Macro-F1 on the strict test split. PhoBERT achieves the best offline routing accuracy but at higher inference cost (53.3 ms vs. 8.9 ms for XGBoost).

Latency also supports the routing motivation. Single-stage Router is slightly faster than Pure Graph (2,209.2 ms vs. 2,283.4 ms) while producing higher F1. This result is important be-

cause the project is not trying to always use GraphRAG; it is trying to avoid graph traversal when dense retrieval is sufficient, while still preserving graph and hybrid routes for relation-heavy questions.

5.2 Clarification Benchmark

Table 12 reports the original pre-Phase-3 configuration on the 234-query constructed ambiguity benchmark. The most important finding is that Stage 1 has clarify F1 = 0.000. This is not an implementation bug. Stage 1 is trained on the strict three-class dataset with labels `dense_retrieval`, `graph_traversal`, and `hybrid_reasoning`; it has no `clarify` class during training and therefore cannot architecturally predict clarification. In this initial configuration, clarification behavior depends on Stage 2 and the ambiguity trigger.

Table 12: Original pre-Phase-3 clarification results on the constructed ambiguity benchmark.

Variant	Route	Acc.	P	R	F1	Trigger
Stage 1 only		0.269	0.000	0.000	0.000	0.000
Stage 1 + Stage 2		0.585	1.000	0.519	0.684	0.577

The initial two-stage configuration improves clarify F1 from 0.000 to 0.684. Its clarify precision is 1.000, meaning that when the system predicts `clarify`, the prediction is correct in this benchmark. However, clarify recall is only 0.519, so many ambiguous queries are still missed. This pattern is preferable to an over-clarifying system in a first iteration because it avoids false clarification, but it also shows that the initial ambiguity detector is incomplete.

Table 13 explains the failure mode. Stage 2 detects surface-context ambiguity very well: unresolved pronoun references and incomplete context both have 100% trigger rate and 100% re-

call. However, it completely misses missing-entity ambiguity and multiple-interpretation ambiguity in the current benchmark. These two categories require more semantic understanding because the query can look grammatically complete while still lacking the specific legal target or allowing multiple valid legal interpretations.

Table 13: Stage 2 behavior by ambiguity type.

Ambiguity type	Total	Trigger	Recall
Pronoun reference	42	1.000	1.000
Incomplete context	39	1.000	1.000
Missing entity	39	0.000	0.000
Multi-interpretation	36	0.000	0.000

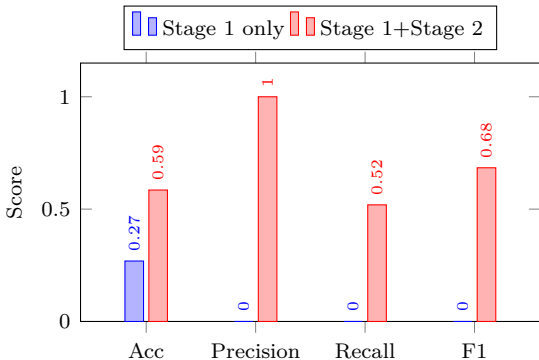


Figure 4: Original pre-Phase-3 clarification metrics on the ambiguity benchmark. Stage 1 scores zero on all clarification metrics because it has no clarify label. Stage 2 achieves perfect precision (1.000) by relying on explicit surface-context cues.

5.3 Conversation-aware Ambiguity Stress Test

After adding deterministic history-referent resolution and a severe-ambiguity override policy, we evaluate the router on the 160-query conversation stress test described in Section 3.2.

This diagnostic is reported separately from the strict end-to-end benchmark because it measures whether the router uses conversation history correctly, not answer generation quality. Table 14 compares the router before and after Phase 3. Clarify F1 improves from 0.545 to 0.862, and history-resolution accuracy reaches 0.850. The improvement comes from two complementary behaviors: resolved history can unblock retrieval, while absent, irrelevant, or conflicting history can preserve or force clarification.

Table 14: Diagnostic conversation-aware ambiguity stress-test results.

Variant	Route	Acc.	P	R	F1	Hist.	Acc.
Before Phase 3	0.475	0.907	0.390	0.545			n/a
After Phase 3	0.750	0.963	0.780	0.862			0.850

Table 15 reports the after-Phase 3 behavior by category. Clear dense controls remain intact with route accuracy 1.000 and a low Stage 2 rate, showing that the dense fast path is preserved. The router also improves clarification for no-history, irrelevant history, conflicting history, missing-entity, and multi-interpretation cases. However, answerable follow-up questions and graph/hybrid controls remain difficult because the router may resolve that a query is answerable but still confuse the correct retrieval mechanism.

The original 234-query ambiguity benchmark was also rerun after adding deterministic history-referent resolution and the severe-ambiguity override, as shown in Table 16. Clarify F1 increases from 0.684 to 0.840 with zero false positives. The strongest gains are on missing-entity cases, but multi-interpretation remains weak on the original template benchmark because these queries often look answerable without an explicit contextual pronoun.

Qualitatively, the demo shows the intended

Table 15: Diagnostic conversation stress-test category results after Phase 3.

Category	Score	S2	Interpretation
Answerable history	0.600	0.850	Remaining dense/graph/hybrid confusion.
No-history clarify	0.850	1.000	No-history references mostly clarified.
Irrelevant history	0.750	0.900	Irrelevant history no longer suppresses ambiguity.
Conflicting history	0.700	0.900	Multiple candidates remain difficult.
Missing entity	0.850	1.000	Missing legal targets are better detected.
Multi-interpret.	0.750	0.850	Improved on the conversation stress test.
Dense control	1.000	0.150	Dense fast path is preserved.
Graph/hybrid ctrl.	0.500	0.950	Relation-heavy route choice remains hard.

Table 16: Original ambiguity benchmark before and after deterministic history-referent resolution and severe-ambiguity override.

Variant	Acc.	P	R	F1	FP
Before Phase 3	0.585	1.000	0.519	0.684	0
After Phase 3	0.722	1.000	0.724	0.840	0

behavior. With history mentioning “*Nghị định 100/2019/NĐ-CP*”, the follow-up query “*Văn bản đó còn hiệu lực không?*” is resolved to that decree and routed to graph traversal. With empty or irrelevant history, the same surface query is routed to `clarify` instead of retrieving evidence for an arbitrary document.

5.4 Stage 2 Cost and Override Behavior

The benefit of Stage 2 comes with a substantial latency cost. In the strict full benchmark, Two-stage Hybrid triggers Stage 2 for 43.0% of queries and changes the Stage 1 route for only 4.26% of triggered cases. In the ambiguity benchmark, Stage 2 is triggered for 57.7% of queries and has a much higher override rate of 69.6%. These two numbers are not contradictory. The strict benchmark contains mostly answerable retrieval questions, whereas the ambiguity benchmark intentionally contains many underspecified queries.

Routing-only latency measurements show the same pattern. Stage 1-only routing costs approximately 33.7 ms on 942 non-Stage 2 cases, while routing with Stage 2 costs approximately 4,017 ms on 258 Stage 2 cases. This is a roughly 119× latency increase. Therefore, invoking Stage 2 for every query would undermine the efficiency goal of adaptive routing. The correct design is selective verification: Stage 1 should handle clear cases, while Stage 2 should be reserved for ambiguous, uncertain, or relation-heavy queries.

The Phase 3 conversation stress test confirms this trade-off. Clarification quality improves, but Stage 2 is triggered for 82.5% of conversation-stress queries and average routing latency is 3,127.2 ms. A separate routing-only strict sanity run after Phase 3 reaches routing accuracy 0.8933 with 10 false clarifications out of 600 queries. Because this sanity run does not

perform retrieval or answer generation, and because it is lower than the full strict end-to-end routing result in Table 9, it is not used to replace the main strict benchmark results.

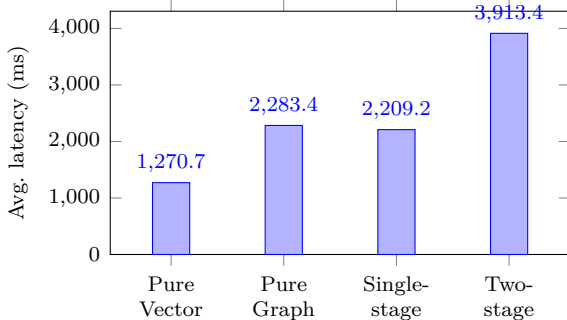


Figure 5: Average end-to-end latency per query. Single-stage Router is faster than Pure Graph while achieving higher F1, confirming that adaptive routing reduces unnecessary graph traversal. Two-stage Hybrid latency reflects the 43% Stage 2 trigger rate.

5.5 Threshold Ablation

Table 17 summarizes the threshold ablation on the development setting. Sweeping the Stage 1 confidence threshold τ_{conf} from 0.50 to 0.99 does not change routing accuracy, which remains 0.7529. It only changes Stage 2 trigger rate. This indicates that many Stage 1 predictions are already high confidence, so τ_{conf} is not the main bottleneck in this setting.

The clarification override threshold τ_{clar} has a stronger effect. A low value such as 0.50 is too aggressive on non-ambiguous development data and reduces routing accuracy to 0.6839. Increasing τ_{clar} to 0.95 improves routing accuracy to 0.8448. This result should be interpreted carefully: the ablation is performed on a routing-oriented development set, not on the ambiguity

Table 17: Selected threshold ablation results.

Parameter	Value	Acc.	Stage2 rate
τ_{conf}	0.50	0.7529	0.2299
τ_{conf}	0.70	0.7529	0.3046
τ_{conf}	0.95	0.7529	0.5920
τ_{conf}	0.99	0.7529	0.5977
τ_{clar}	0.50	0.6839	0.2299
τ_{clar}	0.70	0.7529	0.2299
τ_{clar}	0.90	0.7759	0.2299
τ_{clar}	0.95	0.8448	0.2299

benchmark. Therefore, the threshold that maximizes strict routing accuracy may not maximize clarify recall. The practical conclusion is that τ_{clar} must be calibrated jointly for both false clarification on answerable queries and missed clarification on ambiguous queries.

5.6 Overall Discussion

The experiments support the main thesis of the project: Vietnamese legal QA should not use a fixed retrieval strategy for every query. Dense retrieval is strong for direct lookup, but it is weak when the answer depends on explicit legal relations. Graph traversal is useful for relation-heavy queries, but applying it to every question can add cost and produce diffuse context. Hybrid reasoning is appropriate when both relation paths and article text are needed. Clarification is necessary when the legal target is underspecified.

The current strongest result is not that Stage 2 always improves every metric. Instead, the stronger and more honest finding is that Stage 1 is already sufficient for many strict retrieval-routing cases, while Stage 2 is necessary for clarification because Stage 1 has no clarify label. The clarify benchmark reveals a precise limitation: Stage 2 recognizes surface context-

tual ambiguity but misses semantic ambiguity. This finding is useful for the next development step because it points directly to missing-entity detection, entity-linking uncertainty, and multi-interpretation generation as the components that should be improved.

The conversation-aware stress test strengthens this analysis. The router no longer treats conversation history as a binary feature. Instead, it distinguishes resolved history from absent, irrelevant, and conflicting history. This allows a follow-up query with a unique referent to proceed to retrieval, while the same query without a usable referent can ask for clarification. The trade-off is higher Stage 2 usage and latency on conversation-stress examples. Moreover, the routing-only strict sanity run after Phase 3 still produces 10 false clarifications, so the Phase 3 diagnostic results should be interpreted as an ambiguity improvement rather than a replacement for the strict end-to-end QA table.

6 Limitations and Future Work

Several limitations remain. First, the strict training split does not include a `clarify` class, so Stage 1 cannot predict clarification by design. The initial Stage 2 configuration improves clarify F1 from 0.000 to 0.684, and the Phase 3 additions improve the original ambiguity benchmark further to 0.840 and the separate conversation stress test to 0.862. However, clarification still depends on the LLM verifier and remains sensitive to trigger calibration. Second, the ambiguity benchmarks are synthetic and template-generated. They are useful for controlled stress testing, but they should be validated with manually written or human-reviewed ambiguous legal questions before being treated as realistic production benchmarks. Third, the initial configuration detects surface cues such as unre-

solved pronouns and incomplete context more reliably than semantic ambiguity. After Phase 3, missing-entity cases improve substantially, but multi-interpretation remains weak on the original template benchmark.

Fourth, Stage 2 introduces a large latency cost. Routing with Stage 2 is about $119\times$ slower than Stage 1-only routing in the measured routing logs. Therefore, the verifier should not be used for every query. Future work should calibrate the trigger function with both strict routing and ambiguity benchmarks, rather than optimizing only one setting. Fifth, strict-test routing accuracy decreases slightly from 0.9350 to 0.9283 when Stage 2 is enabled. This happens because the strict benchmark contains no intended clarify queries, so some Stage 2 interventions are unnecessary in that setting.

The Phase 3 conversation-aware extension introduces additional limitations. Although conversation clarify F1 improves to 0.862, `answerable_with_history` route accuracy is only 0.600, meaning that the system can resolve that a query is answerable but still confuse dense, graph, and hybrid execution. The clear graph/hybrid control category also remains difficult with route accuracy 0.500. On the original template ambiguity benchmark, `multi_interpretation` remains weak even after Phase 3, despite improvements on missing-entity and pronoun-reference cases. Finally, a routing-only strict sanity run after Phase 3 still produces 10 false clarifications out of 600 queries; therefore, these diagnostic improvements are not used to replace the strict end-to-end metrics in Table 9.

Sixth, the graph is larger and better connected after the migration, but curated legal relations remain sparse: AMENDS, REPEALS, and GUIDES have far fewer edges than generic

REFERENCES. As a result, graph traversal may still miss important legal relations or return overly broad reference context. Seventh, the primary routing labels are derived from hop-count and cross-document metadata rather than independent human annotation. These labels are useful for training and benchmarking, but manual review is needed to validate whether the assigned route truly matches the best retrieval strategy. Finally, end-to-end answer quality is evaluated with automatic string-based metrics, which are imperfect for legal answers. Future evaluation should include human judgments of legal correctness, evidence faithfulness, citation usefulness, and whether clarification questions are appropriate.

Future work will focus on integrating manually reviewed `clarify` samples into the main training set, improving semantic ambiguity detection through entity-linking uncertainty and candidate-interpretation generation, enriching the legal graph with more reliable amendment and repeal relations, replacing the XGBoost Stage 1 classifier with a PhoBERT-based model to improve routing Macro-F1 from 0.739 to approximately 0.901 at the cost of higher inference latency, applying class-balancing techniques to address the training-set imbalance toward `dense_retrieval`, and expanding answer-quality evaluation with human review.

7 Conclusion

This paper presented a Reasoning-aware Adaptive Router for Vietnamese Legal Hybrid GraphRAG. The system combines a dense vector index, a Neo4j legal knowledge graph, a Stage 1 XGBoost router, and a selective Stage 2 LLM verifier. Instead of always using dense retrieval or always using graph traversal, the router selects among dense retrieval, graph traversal, hy-

brid reasoning, and clarification according to the query’s reasoning demand and ambiguity level.

The strict 600-query benchmark shows that adaptive routing improves answer F1 over both Pure Vector and Pure Graph. The Single-stage Router reaches $F1 = 0.4231$, compared with 0.3626 for Pure Vector and 0.3556 for Pure Graph. The Two-stage Hybrid system obtains the best F1, 0.4235, but has higher latency and slightly lower strict routing accuracy than the single-stage system. This confirms that Stage 2 should be used selectively rather than as a default step for every query.

The ambiguity benchmarks show that clarification handling is the main role of Stage 2. Stage 1 has clarify $F1 = 0.000$ because it is trained only on three retrieval labels. The initial Stage 2 configuration improves clarify F1 to 0.684 with precision 1.000. After adding deterministic history-referent resolution and the severe-ambiguity override, the original ambiguity benchmark improves to $F1 = 0.840$. The Phase 3 conversation-aware extension further shows that history resolution is a useful intermediate signal for legal QA: on the separate 160-query conversation stress test, the router reaches clarify $F1 = 0.862$ and history-resolution accuracy $= 0.850$.

The same follow-up query can therefore be routed differently depending on whether history contains a unique legal referent. However, multi-interpretation ambiguity remains weak on the original template benchmark, resolved-history route selection is still imperfect, and the strict routing-only sanity check still produces false clarifications. These results support the central design of reasoning-aware adaptive routing while showing that the Phase 3 diagnostic improvements should not replace the strict end-to-end configuration without further calibration.

Acknowledgements

The author thanks the School of Information Technology and Communication at Hanoi University of Science and Technology for supporting this research. The author is grateful to supervisors Pham Van Hai and Nguyen Viet Cuong for their guidance throughout the project.

References

- [1] Asai, A., Wu, Z., Wang, Y., Sil, A., Hajishirzi, H.: Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In: The Twelfth International Conference on Learning Representations (2024)
- [2] Chalkidis, I., Fergadiotis, M., Malakasiotis, P., Aletras, N., Androutsopoulos, I.: LEGAL-BERT: The muppets straight out of law school. In: Findings of the Association for Computational Linguistics: EMNLP 2020, pp. 2898–2904. Association for Computational Linguistics (2020)
- [3] Chen, L., Zaharia, M., Zou, J.: FrugalGPT: How to use large language models while reducing cost and improving performance. In: Workshop on Efficient Systems for Foundation Models at ICML 2023 (2023)
- [4] Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794 (2016)
- [5] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Larson, J.: From local to global: A graph RAG approach to query-focused summarization. arXiv preprint arXiv:2404.16130 (2024)
- [6] Elgohary, A., Peskov, D., Boyd-Graber, J.: Can you unpack that? learning to rewrite questions-in-context. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, pp. 5918–5924. Association for Computational Linguistics (2019)
- [7] Guu, K., Lee, K., Tung, Z., Pasupat, P., Chang, M.W.: REALM: Retrieval-augmented language model pre-training. In: Proceedings of the 37th International Conference on Machine Learning, pp. 3929–3938. PMLR (2020)
- [8] Jeong, S., Baek, J., Cho, S., Hwang, S.J., Park, J.C.: Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pp. 7029–7043. Association for Computational Linguistics (2024)
- [9] Jiang, Z., Xu, F.F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J., Neubig, G.: Active retrieval augmented generation. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pp. 7969–7992. Association for Computational Linguistics (2023)
- [10] Jiménez Gutiérrez, B., Shu, Y., Gu, Y., Yasunaga, M., Su, Y.: HippoRAG: Neurobiologically inspired long-term memory for large language models. arXiv preprint arXiv:2405.14831 (2024)

- [11] Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W.t.: Dense passage retrieval for open-domain question answering. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 6769–6781. Association for Computational Linguistics (2020)
- [12] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W.t., Rocktaschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474 (2020)
- [13] Li, S., He, Y., Guo, H., Bu, X., Bai, G., Liu, J., Liu, J., Qu, X., Li, Y., Ouyang, W., Su, W., Zheng, B.: Graphreader: Building graph-based agent to enhance long-context abilities of large language models. In: *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics (2024)
- [14] Min, S., Michael, J., Hajishirzi, H., Zettlemoyer, L.: AmbigQA: Answering ambiguous open-domain questions. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 5783–5797. Association for Computational Linguistics (2020)
- [15] Nguyen, D.Q., Nguyen, A.T.: PhoBERT: Pre-trained language models for vietnamese. In: *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1037–1042. Association for Computational Linguistics (2020)
- [16] Ong, I., Arumugam, D., Demszky, D., Yang, D.: RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665* (2024)
- [17] Press, O., Zhang, M., Min, S., Schmidt, L., Smith, N.A., Lewis, M.: Measuring and narrowing the compositionality gap in language models. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 5687–5711. Association for Computational Linguistics (2023)
- [18] Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., Manning, C.D.: RAPTOR: Recursive abstractive processing for tree-organized retrieval. In: *The Twelfth International Conference on Learning Representations* (2024)
- [19] Sovrano, F., Palmirani, M., Vitali, F.: Legal knowledge graphs for explainable artificial intelligence. In: *Legal Knowledge and Information Systems*, pp. 175–184 (2020)
- [20] Sun, H., Bedrax-Weiss, T., Cohen, W.W.: Pullnet: Open domain question answering with iterative retrieval on knowledge bases and text. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pp. 2380–2390. Association for Computational Linguistics (2019)
- [21] Trivedi, H., Balasubramanian, N., Khot, T., Sabharwal, A.: Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. *arXiv preprint arXiv:2212.10509* (2023)
- [22] Vu, T., Nguyen, D.Q., Nguyen, D.Q., Dras, M., Johnson, M.: VnCoreNLP: A viet-

- nameuse natural language processing toolkit. In: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations, pp. 56–60. Association for Computational Linguistics (2018)
- [23] Vu, T., Nguyen, Y., Nguyen, H., Pham, C., Tran, C.: MIST: A multilingual dataset and benchmark for fine-grained audio inpainting tampering localization. *IEEE Access* **14**, 76,700–76,721 (2026). DOI 10.1109/ACCESS.2026.3694045
- [24] Wang, Y., Lipka, N., Rossi, R.A., Siu, A.F., Zhang, R., Derr, T.: Knowledge graph prompting for multi-document question answering. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, pp. 19,206–19,214 (2024)
- [25] Zhong, H., Xiao, C., Tu, C., Zhang, T., Liu, Z., Sun, M.: How does NLP benefit legal system: A summary of legal artificial intelligence. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp. 5218–5230. Association for Computational Linguistics (2020)